

AI Tech Lead Assignment

Context

You will be (imagine) leading a small squad that is building an **AI-powered Document Insight Platform** for internal customers. Your mission in this exercise is to design, implement, and document a production-ready slice of that platform that can:

1. Ingest unstructured documents (PDFs and images).
2. Extract and store structured representations.
3. Serve question-answering and semantic search over the stored content.
4. Run reliably in a cloud-native environment and be easy for a team to extend.

Treat this as a green-field initiative whose first release must already demonstrate good engineering discipline.

What to Deliver

Deliverable	Required	Notes
Working code in a public GitHub repository	✓	Core services must be written in Python. Subsidiary components (e.g. infrastructure scripts, front-end demo) may use any language.
Architecture document (docs/architecture.md)	✓	System diagram, component responsibilities, data-flow, and design trade-offs.
1–3 Architectural Decision Records (docs/adr/)	✓	Each ≈ 1 page; format is up to you.
CI/CD pipeline	✓	Build, test, security-scan, and publish Docker images (or equivalent) on every push.
Observability hook-up	○	At least one metric and one distributed trace spanning a user request.
Load-test script & short report	○	Show how the service behaves at ~100 requests/s.
README	✓	Local setup, Docker/K8s instructions, sample API calls, design summary.
Team-level extras (CONTRIBUTING, CODEOWNERS, pre-commit, etc.)	○	Optional, but demonstrates leadership culture.

☑ - REQUIRED

○ - OPTIONAL

Functional Requirements

1. Document Ingestion Service

Endpoint: POST /ingest

- Accepts PDFs or images (≥ 10 MiB each).
- Stores originals in object storage.
- Emits an event for downstream processing (choose your preferred mechanism).

2. Processing Worker

- Extracts text via OCR / PDF parsing.
- Detects language and runs Named Entity Recognition.
- Generates embeddings and stores them in a vector store (choose any; e.g. FAISS).

3. Query Service

Endpoint: POST /query with question, filter, and top_k.

- Retrieves relevant passages and produces an answer (extractive QA or retrieval-augmented generation). Use any LLM API (i.e. Mistral offers free tokens)
- Returns: answer, confidence score, quoted sources, and detected entities.

4. Gateway / BFF

- Single public API that performs authentication (e.g. JWT) and per-user rate-limiting.

5. Multi-tenancy & Security

- Isolate data between tenants (schema, table, or row level).
- Encrypt data in transit and at rest.

Engineering Excellence Requirements

Area	Expectations
Code Quality	Idiomatic Python, type hints, ≥ 70 % unit-test coverage, meaningful docstrings.
Testing Pyramid	Unit tests, service-level integration tests (docker-compose or equivalent), and one end-to-end scenario.
Performance	A benchmark/ folder containing a load script and a brief results summary.
Observability	Metrics and traces that let an engineer spot slow stages in the pipeline.
CI/CD	On every push: lint, tests, security scan. On main: build & push container images (or similar artefacts).
Documentation	Clear architecture doc, ADRs, and a concise README for developers and operators.
Leadership Signals	Evidence of coding standards, contribution guidelines, and thoughtful commit history.

Optional Stretch Ideas

(Pick any that showcase your strengths. Novel alternatives are also welcome)

- **Scalability:** autoscaling rules; sharding strategy for the vector store.
- **MLOps:** experiment tracking, model registry, or canary/B-testing workflow.
- **Privacy / Compliance:** automatic redaction of PII; audit logging.
- **Cost Awareness:** rough \$ / 1 000 documents estimate for the proposed stack.
- **Frontend:** a small demo UI to visualise ingestion and QA results.
- **Edge Cases:** duplicate-document detection; multilingual support.

Evaluation Criteria

Area	Focus
System Design	Scalability, fault-tolerance, clear trade-offs.
Technical Depth	Correct use of AI/ML techniques; clean service boundaries.
Code Craftsmanship	Readability, tests, error-handling, extensibility.
DevOps & Reliability	CI/CD completeness, observability, infra as code.
Leadership Qualities	Clarity of docs, ADR discipline, team-oriented tooling.
Creativity	Elegant extras, thoughtful tooling choices, clear performance insights.

Submission Checklist

1. ☒ Clone, run make local-run, and confirm API works.
2. ☒ README explains local, Docker, and (optionally) K8s setups.
3. ☒ pytest suite passes.
4. ☒ Architecture diagram rendered in docs/.
5. ☒ Metrics/traces demonstrable via screenshots or a live link.
6. ☒ CI badge shows green on main.
7. ☒ (Optional) Short demo video (≤ 3 min).

We're excited to see how you approach building a production-ready AI service.

Good luck!